Abstract Factory

Season: 1 · Creational · Category: GoF · Level: Intermediate

Intent

- Create families of related products without naming concrete classes.

Problem

- Without this pattern, code tends toward: New things up with conditionals and duplicated construction.

Real-world Analogy

- IKEA room kits: Scandinavian or Industrial — sofa and lamp match the theme.

When to Use

- UI themes, cloud provider SDKs, multi-brand product lines.
- The design force named in Intent shows up more than once.

When Not to Use

- Only one product type; Factory Method is enough.
- A simpler language feature (record, sealed switch, function value) already solves it.

Pattern Structure

- See mermaid diagram in explanation.md

Classic Implementation

```java
package com.buildwithwaqar.patterns.gof.creational.abstractfactory;

interface Button { String paint(); }
interface Checkbox { String paint(); }

record LightButton() implements Button {
    public String paint() { return "light-button"; }
}
record DarkButton() implements Button {
    public String paint() { return "dark-button"; }
}
record LightCheckbox() implements Checkbox {
    public String paint() { return "light-checkbox"; }
}
record DarkCheckbox() implements Checkbox {
    public String paint() { return "dark-checkbox"; }
}

/** Family of related products without specifying concrete classes. */
public interface UiFactory {
    Button button();
    Checkbox checkbox();

    static UiFactory of(Theme theme) {
        return switch (theme) {
            case LIGHT -> new LightFactory();
            case DARK -> new DarkFactory();
        };
    }
```

Classic Implementation (2)

```
}

enum Theme { LIGHT, DARK }

record LightFactory() implements UiFactory {
    public Button button() { return new LightButton(); }
    public Checkbox checkbox() { return new LightCheckbox(); }
}

record DarkFactory() implements UiFactory {
    public Button button() { return new DarkButton(); }
    public Checkbox checkbox() { return new DarkCheckbox(); }
}

record Dialog(UiFactory factory) {
    String render() {
        return factory.button().paint() + "+" + factory.checkbox().paint();
    }
}
```

Modern Java 25 / Functional

- UiFactory.of(Theme) + records for Light/Dark products.

Worked Example

```
See the JUnit test beside the implementation.
Run: mvn -Dtest=*Test test
Focus package matching: gof/creational/abstractfactory/
```

Pitfalls

- Keep families consistent.
- Wire with DI.

Interview Q&A

- Q: How do you test Abstract Factory? A: Inject a fake factory returning test doubles for the whole family.
- Q: How would you test this? A: Prefer injecting interfaces or interpreters; keep demos deterministic (fixed clocks, canned inputs).

Key Takeaways

- Create families of related products without naming concrete classes.
- Prefer Java 25 sealed types / records / virtual threads where they simplify the classic shape.
- Ship working tests with every episode.

Video Script Highlights

- [0:00] Hook — Open with the analogy: IKEA room kits: Scandinavian or Industrial — sofa and lamp match the theme. [0:40] Intent — State th...